Частное образовательное учреждение высшего образования
«Санкт-Петербургский институт экономики и управления»
(СПбИЭУ)

Специальность 09.02.07 «Информационные системы и программирование»
ПМ.03 Сопровождение и обслуживание программного обеспечения компьютерных систем
МДК.03.01 Внедрение и поддержка компьютерных систем

ОТЧЁТ

по промежуточному контролю № 20

Тема 1.1. Основные методы внедрения и анализа функционирования ПО

«Мониторинг доступности внешнего API и отправка структурированного лога»

Вариант 10

Выполнил: студент группы И14-23-9
Иванов Егор Михайлович

Проверила: преподаватель
Чудинова Александра Анатольевна
Оценка: _____

Санкт-Петербург
2026

1. Описание задания

Задача: разработать консольное приложение на C# (.NET 8) для АРМ дежурного инженера. Программа проверяет доступность публичного сервиса, фиксирует время отклика (Latency), логирует событие через **Serilog** в локальный файл и имитирует отправку метрики. Основа — эталонный пример `Program.cs` из задания (Serilog: консоль + файл `C:\SmartCampus\Logs\arm_monitor_log.txt` с ежедневной ротацией; Stopwatch; HttpClient с заголовком User-Agent; try-catch-finally с `Log.CloseAndFlush()`).

Таблица 1 – Индивидуальный вариант

Вариант	**Название задачи АРМ**	**Бизнес-цель и задача**	**Публичный источник / API**
10	Экспортер ведомости питания столовой	Симулировать выгрузку транзакций питания, сгруппировать данные по категориям льгот и сформировать отчётный файл в формате CSV	https://jsonplaceholder.typicode.com/posts

Требования теории (занятие № 20), учтённые в программе

Таблица 2 – Связь с теорией

Тема лекции	**Как реализовано**
Логирование через Serilog, Dual Sinks: локальный файл для офлайн-аудита + канал для оповещений	WriteTo.Console + WriteTo.File (RollingInterval.Day), единый outputTemplate
Метка конкретного АРМ: Enrich.WithProperty("StationId", …)	StationId из appsettings.json / CAMPUS_STATION_ID в каждой строке лога
Структурированное логирование — параметры, а не склеенная строка	все события с именованными свойствами {StatusCode}, {ElapsedMs}, {Category}…
Трейсинг: каждая операция получает TraceId	Activity с W3C TraceId, заголовок traceparent в запросе, TraceId в логе, CSV и метрике
Иерархия конфигурации: appsettings.json → Environment Variables → аргументы	ArmSettings: appsettings.json → CAMPUS_STATION_ID, CAMPUS_TARGET_URL → --url
Мониторинг: администратор узнаёт о сбое раньше звонка вахтёра	метрика arm_api_up / arm_api_latency_ms (имитация OTLP), порог деградации 2000 мс, коды завершения

2. Программный код с комментариями

Решение `CampusArmMonitor.sln` состоит из двух файлов: `Program.cs` — эталонная логика мониторинга, расширенная по варианту 10, и `MealStatement.cs` — экспорт ведомости по категориям льгот. NuGet-пакеты: Serilog 4.0.2, Serilog.Sinks.Console 6.0.0, Serilog.Sinks.File 6.0.0.

Исходный код решения и документация в репозитории GitHub: https://github.com/Egorvan606-senior/mdk03/tree/main/pk-20

Листинг 1 – Program.cs

```
// ============================================================================
// CampusArmMonitor — АРМ дежурного инженера ИС «Умный кампус» (.NET 8)
// Промежуточный контроль № 20, вариант 10: «Экспортер ведомости питания столовой».
// Автор: Иванов Е.М., группа И14-23-9.
//
// Программа построена на эталонном примере из задания и расширена по варианту:
//   1. Инициализирует Serilog с двумя приёмниками (Dual Sinks): консоль + локальный
//      файл C:\SmartCampus\Logs\arm_monitor_log<дата>.txt с ежедневной ротацией.
//   2. Проверяет доступность публичного API выгрузки транзакций (jsonplaceholder /posts)
//      и фиксирует время отклика (Latency) через Stopwatch.
//   3. Симулирует выгрузку транзакций питания, группирует их по категориям льгот
//      и сохраняет ведомость в CSV (C:\SmartCampus\Reports).
//   4. Имитирует отправку метрики времени отклика в систему мониторинга.
// Каждая проверка получает уникальный TraceId (W3C) — он есть в логе, в CSV и в метрике,
// поэтому все артефакты одной проверки связываются между собой (цифровой след).
//
// Коды завершения: 0 — API доступен, ведомость сформирована; 1 — сбой API/сети;
//                  2 — API доступен, но время отклика выше порога (деградация).
// ============================================================================

using System;
using System.Diagnostics;
using System.IO;
using System.Net.Http;
using System.Text;
using System.Text.Json;
using System.Threading.Tasks;
using Serilog;
using Serilog.Context;

namespace CampusArmMonitor
{
    internal class Program
    {
        private static async Task<int> Main(string[] args)
        {
            Console.OutputEncoding = Encoding.UTF8;

            // 0. Конфигурация: appsettings.json -> переменные окружения CAMPUS_* ->
аргументы (--url)
            var settings = ArmSettings.Load(AppContext.BaseDirectory, args);

            // 1. Инициализация локального логирования через Serilog (Dual Sinks)
            const string template =
                "{Timestamp:yyyy-MM-dd HH:mm:ss.fff zzz} [{Level:u3}] ({StationId})
{Message:lj} {TraceId}{NewLine}{Exception}";

            Log.Logger = new LoggerConfiguration()
                .MinimumLevel.Debug()
                .Enrich.FromLogContext()                                    //
подхватывает TraceId из LogContext
                .Enrich.WithProperty("StationId", settings.StationId)      // метка
конкретного АРМ
                .WriteTo.Console(outputTemplate: template)                  // канал 1:
экран дежурного
                .WriteTo.File(                                              // канал 2:
локальный файл для офлайн-аудита
                    path: Path.Combine(settings.LogDirectory, "arm_monitor_log.txt"),
                    rollingInterval: RollingInterval.Day,                   // новый файл
каждые сутки
                    outputTemplate: template,
                    encoding: Encoding.UTF8)
                .CreateLogger();

            // Activity = операция проверки; .NET генерирует для неё уникальный W3C
TraceId
            using var activity = new Activity("CanteenStatementExport").Start();
            string traceId = activity.TraceId.ToString();
```

```
            using var _ = LogContext.PushProperty("TraceId", $"trace={traceId}");

            Log.Information("=== APM Мониторинг: Запуск проверки инфраструктуры (v2.0.0,
ПК {Machine}) ===", Environment.MachineName);
            Log.Debug("Конфигурация: источник {Url}, таймаут {Timeout} с, порог задержки
{Threshold} мс",
                settings.TargetUrl, settings.TimeoutSeconds, settings.LatencyWarningMs);

            string targetUrl = settings.TargetUrl;
            var stopwatch = Stopwatch.StartNew();

            using var client = new HttpClient { Timeout =
TimeSpan.FromSeconds(settings.TimeoutSeconds) };
            client.DefaultRequestHeaders.Add("User-Agent", "SmartCampus-ARM-Monitor");
            // Заголовок W3C traceparent: сервер (если поддерживает OpenTelemetry)
продолжит ту же трассу
            client.DefaultRequestHeaders.Add("traceparent", activity.Id);

            int exitCode = 1; // по умолчанию — сбой; меняется только при успешной
проверке
            try
            {
                // 2. Выполнение запроса к публичному API
                HttpResponseMessage response = await client.GetAsync(targetUrl);
                stopwatch.Stop();
                long latency = stopwatch.ElapsedMilliseconds;

                if (response.IsSuccessStatusCode)
                {
                    string jsonContent = await response.Content.ReadAsStringAsync();
                    var posts = MealStatement.Parse(jsonContent);

                    Log.Information("✔️ API доступен. Код {StatusCode}, записей выгрузки:
{Count}, время отклика: {ElapsedMs} мс",
                        (int)response.StatusCode, posts.Count, latency);

                    // 3. Экспорт ведомости питания по категориям льгот
                    var lines = MealStatement.Build(posts);
                    foreach (var line in lines)
                    {
                        // структурированный лог: значения — параметры события, а не
склеенная строка
                        Log.Information("Категория {Code} «{Category}»: транзакций
{Count}, сумма {Amount:N2} ₽, дотация {Subsidy:N2} ₽, к оплате {ToPay:N2} ₽",
                            line.Category.Code, line.Category.Name, line.Transactions,
line.Amount, line.Subsidy, line.ToPay);
                    }

                    string csv = MealStatement.SaveCsv(settings.ReportDirectory, lines,
settings.StationId, traceId);
                    Log.Information("Ведомость сохранена: {CsvPath}", csv);

                    if (latency > settings.LatencyWarningMs)
                    {
                        Log.Warning("⚠️ Деградация: время отклика {ElapsedMs} мс
превышает порог {Threshold} мс", latency, settings.LatencyWarningMs);
                        exitCode = 2;
                    }
                    else
                    {
                        exitCode = 0;
                    }

                    // 4. Имитация отправки метрики в систему мониторинга
                    SendMetric(settings, latency, up: true, traceId);
                }
                else
                {
                    Log.Warning("⚠️ API вернул код ошибки: {StatusCode}, время отклика
{ElapsedMs} мс", response.StatusCode, latency);
                    SendMetric(settings, latency, up: false, traceId);
                    exitCode = 1;
                }
            }
            catch (TaskCanceledException ex)
```

```
            {
                // HttpClient сообщает о таймауте через TaskCanceledException
                Log.Error(ex, "✖ Таймаут: сервер не ответил за {Timeout} с",
settings.TimeoutSeconds);
                SendMetric(settings, stopwatch.ElapsedMilliseconds, up: false, traceId);
                exitCode = 1;
            }
            catch (JsonException ex)
            {
                Log.Error(ex, "✖ API ответил, но данные выгрузки повреждены
(некорректный JSON)");
                exitCode = 1;
            }
            catch (Exception ex)
            {
                Log.Error(ex, "✖ Критическая ошибка соединения с сервером
мониторинга!");
                SendMetric(settings, stopwatch.ElapsedMilliseconds, up: false, traceId);
                exitCode = 1;
            }
            finally
            {
                Log.Information("=== Проверка завершена, код завершения {ExitCode} ===",
exitCode);
                Log.CloseAndFlush(); // гарантированно дописывает буфер в файл лога
            }

            return exitCode;
        }

        /// <summary>
        /// Имитация отправки метрики. В промышленной эксплуатации метрика уходит по OTLP
        /// (порт 4317) в OpenTelemetry Collector; здесь она записывается в журнал метрик
        /// в формате Prometheus и фиксируется в логе Serilog.
        /// </summary>
        private static void SendMetric(ArmSettings settings, long latencyMs, bool up,
string traceId)
        {
            string labels = $"station=\"{settings.StationId}\",endpoint=\"posts\"";
            string payload =
                $"arm_api_up{{{labels}}} {(up ? 1 : 0)}\n" +
                $"arm_api_latency_ms{{{labels}}} {latencyMs}\n";

            try
            {
                Directory.CreateDirectory(settings.LogDirectory);
                File.AppendAllText(Path.Combine(settings.LogDirectory,
"arm_metrics.prom"),
                    $"# {DateTime.Now:O} trace={traceId}\n{payload}");
                Log.Information("Метрика отправлена (имитация OTLP ->
otel.campus.internal:4317): up={Up}, latency={LatencyMs} мс", up ? 1 : 0, latencyMs);
            }
            catch (IOException ex)
            {
                // сбой отправки метрики не должен ронять проверку
                Log.Warning(ex, "Не удалось записать метрику");
            }
        }
    }

    /// <summary>Настройки APM: appsettings.json, переопределяемые переменными окружения
и аргументами.</summary>
    internal sealed class ArmSettings
    {
        public string TargetUrl { get; set; } =
"https://jsonplaceholder.typicode.com/posts";
        public string StationId { get; set; } = "CANTEEN-A1-ARM-01";
        public string LogDirectory { get; set; } = @"C:\SmartCampus\Logs";
        public string ReportDirectory { get; set; } = @"C:\SmartCampus\Reports";
        public int TimeoutSeconds { get; set; } = 10;
        public int LatencyWarningMs { get; set; } = 2000;

        public static ArmSettings Load(string baseDir, string[] args)
        {
            string path = Path.Combine(baseDir, "appsettings.json");
```

```
            var s = File.Exists(path)
                ? JsonSerializer.Deserialize<ArmSettings>(File.ReadAllText(path),
                      new JsonSerializerOptions { PropertyNameCaseInsensitive = true,
ReadCommentHandling = JsonCommentHandling.Skip })
                  ?? new ArmSettings()
                : new ArmSettings();

            // Переменные окружения АРМ (задаёт администратор при развертывании)
            s.StationId = Environment.GetEnvironmentVariable("CAMPUS_STATION_ID") ??
s.StationId;
            s.TargetUrl = Environment.GetEnvironmentVariable("CAMPUS_TARGET_URL") ??
s.TargetUrl;

            // Аргументы командной строки — наивысший приоритет
            for (int i = 0; i + 1 < args.Length; i++)
            {
                if (args[i] == "--url") s.TargetUrl = args[i + 1];
            }

            // Вне Windows (CI/Linux-стенд) путь C:\ недоступен — пишем рядом с
программой
            if (!OperatingSystem.IsWindows())
            {
                s.LogDirectory = Path.Combine(baseDir, "SmartCampus", "Logs");
                s.ReportDirectory = Path.Combine(baseDir, "SmartCampus", "Reports");
            }

            return s;
        }
    }
}
```

Листинг 2 – MealStatement.cs — ведомость питания по категориям льгот

```
using System.Globalization;
using System.Text;
using System.Text.Json;

namespace CampusArmMonitor
{
    /// <summary>Запись мок-сервера jsonplaceholder (/posts) — строка выгрузки кассы
столовой.</summary>
    public sealed record SourcePost(int UserId, int Id, string Title, string Body);

    /// <summary>Категория льготы на питание и доля стоимости, которую покрывает
дотация.</summary>
    public sealed record BenefitCategory(string Code, string Name, decimal SubsidyRate);

    /// <summary>Строка ведомости — итоги по одной категории льгот.</summary>
    public sealed record StatementLine(BenefitCategory Category, int Transactions,
decimal Amount, decimal Subsidy)
    {
        public decimal ToPay => Amount - Subsidy;
        public decimal AverageCheck => Transactions == 0 ? 0 : Math.Round(Amount /
Transactions, 2);
    }

    /// <summary>
    /// Экспортер ведомости питания: симулирует транзакции по выгрузке,
    /// группирует их по категориям льгот и сохраняет ведомость в CSV.
    /// </summary>
    public static class MealStatement
    {
        // Справочник льгот столовой кампуса (учебные данные)
        public static readonly BenefitCategory Full = new("L100", "Полная льгота",
1.00m);
        public static readonly BenefitCategory LowIncome = new("L050", "Малоимущие
семьи", 0.50m);
        public static readonly BenefitCategory Athletes = new("L030", "Спортсмены
сборной", 0.30m);
        public static readonly BenefitCategory None = new("L000", "Без льготы", 0.00m);

        private static readonly CultureInfo Ru = CultureInfo.GetCultureInfo("ru-RU");
```

```
        /// <summary>Разбор JSON-ответа API в список записей.</summary>
        public static List<SourcePost> Parse(string json) =>
            JsonSerializer.Deserialize<List<SourcePost>>(json, new
JsonSerializerOptions(JsonSerializerDefaults.Web))
            ?? throw new JsonException("API вернул пустой ответ");

        /// <summary>Категория льготы покупателя: определяется по userId записи (имитация
справочника студентов).</summary>
        public static BenefitCategory CategoryOf(SourcePost post) => post.UserId switch
        {
            <= 2 => Full,
            <= 4 => LowIncome,
            5 => Athletes,
            _ => None
        };

        /// <summary>Стоимость покупки: комплекс (обед 185 ₽) + 0–4 доп. позиции по 15
₽.</summary>
        public static decimal AmountOf(SourcePost post) => 185m + (post.Body?.Length ??
0) % 5 * 15m;

        /// <summary>Группировка транзакций по категориям льгот (порядок — по убыванию
льготы).</summary>
        public static List<StatementLine> Build(IEnumerable<SourcePost> posts) =>
            posts
                .Where(p => p.Id > 0)
                .GroupBy(CategoryOf)
                .Select(g =>
                {
                    decimal amount = g.Sum(AmountOf);
                    decimal subsidy = Math.Round(g.Sum(p => AmountOf(p) *
g.Key.SubsidyRate), 2);
                    return new StatementLine(g.Key, g.Count(), amount, subsidy);
                })
                .OrderByDescending(l => l.Category.SubsidyRate)
                .ToList();

        /// <summary>
        /// Сохранение ведомости в CSV (разделитель «;», UTF-8 с BOM — открывается в
Excel).
        /// Первая строка — реквизиты отчёта: АРМ, дата, TraceId операции (цифровой
след).
        /// </summary>
        public static string SaveCsv(string directory, List<StatementLine> lines, string
stationId, string traceId)
        {
            Directory.CreateDirectory(directory);
            string path = Path.Combine(directory,
$"meal_statement_{DateTime.Now:yyyyMMdd_HHmmss}.csv");

            var sb = new StringBuilder();
            sb.AppendLine($"Ведомость учёта питания по категориям льгот;АРМ {stationId};
{DateTime.Now:dd.MM.yyyy HH:mm};TraceId {traceId}");
            sb.AppendLine("Код;Категория льготы;Доля дотации;Транзакций;Сумма;Дотация;К
оплате;Средний чек");
            foreach (var l in lines)
            {
                sb.AppendLine(string.Join(';', l.Category.Code, l.Category.Name,
l.Category.SubsidyRate.ToString("P0", Ru),
                    l.Transactions, Money(l.Amount), Money(l.Subsidy), Money(l.ToPay),
Money(l.AverageCheck)));
            }

            sb.AppendLine(string.Join(';', "", "ИТОГО", "", lines.Sum(l =>
l.Transactions),
                Money(lines.Sum(l => l.Amount)), Money(lines.Sum(l => l.Subsidy)),
Money(lines.Sum(l => l.ToPay)), ""));

            File.WriteAllText(path, sb.ToString(), new UTF8Encoding(true));
            return path;
        }

        private static string Money(decimal v) => v.ToString("0.00", Ru);
    }
}
```

Листинг 3 – appsettings.json

```
{
  // Публичный источник по варианту 10: имитация выгрузки транзакций кассы столовой
  "TargetUrl": "https://jsonplaceholder.typicode.com/posts",
  // Идентификатор АРМ (переопределяется переменной окружения CAMPUS_STATION_ID)
  "StationId": "CANTEEN-A1-ARM-01",
  // Папки логов и ведомостей. На Windows — C:\SmartCampus\... как в эталонном примере
  "LogDirectory": "C:\\SmartCampus\\Logs",
  "ReportDirectory": "C:\\SmartCampus\\Reports",
  "TimeoutSeconds": 10,
  // Порог деградации: если ответ дольше — предупреждение дежурному
  "LatencyWarningMs": 2000
}
```

Листинг 4 – CampusArmMonitor.csproj — пакеты Serilog

```
<Project Sdk="Microsoft.NET.Sdk">

  <!--
    ПК № 20, вариант 10. ИС «Умный кампус».
    АРМ дежурного инженера столовой: мониторинг доступности API выгрузки транзакций,
    экспорт ведомости питания по категориям льгот (CSV), структурированный лог Serilog.
  -->
  <PropertyGroup>
    <OutputType>Exe</OutputType>
    <TargetFramework>net8.0</TargetFramework>
    <Nullable>enable</Nullable>
    <ImplicitUsings>enable</ImplicitUsings>
    <RootNamespace>CampusArmMonitor</RootNamespace>
    <AssemblyName>CampusArmMonitor</AssemblyName>
    <Version>2.0.0</Version>
  </PropertyGroup>

  <ItemGroup>
    <!-- Serilog: ядро + два приёмника (Dual Sinks): консоль и локальный файл с ротацией
-->
    <PackageReference Include="Serilog" Version="4.0.2" />
    <PackageReference Include="Serilog.Sinks.Console" Version="6.0.0" />
    <PackageReference Include="Serilog.Sinks.File" Version="6.0.0" />
  </ItemGroup>

  <ItemGroup>
    <None Update="appsettings.json" CopyToOutputDirectory="PreserveNewest" />
  </ItemGroup>

</Project>
```

Таблица 3 – Справочник категорий льгот (учебные данные)

Код	**Категория льготы**	**Доля дотации**	**Покупатели (userId мок-сервера)**
L100	Полная льгота	100 %	1–2
L050	Малоимущие семьи	50 %	3–4
L030	Спортсмены сборной	30 %	5
L000	Без льготы	0 %	6–10

3. Вывод по решению задачи

Разработано консольное приложение АРМ дежурного инженера на C# .NET 8 по эталонному примеру. Программа проверяет доступность публичного API, измеряет время отклика через Stopwatch и пишет

структурированный лог Serilog одновременно в консоль и в локальный файл с ежедневной ротацией (Dual Sinks). Также она имитирует отправку метрик `arm_api_up` и `arm_api_latency_ms`.

По варианту 10 программа выгружает транзакции питания, группирует их по четырём категориям льгот, рассчитывает сумму, дотацию, сумму к оплате и средний чек, и сохраняет ведомость в CSV для Excel. Каждая проверка получает W3C TraceId, который попадает в лог, ведомость и метрику, поэтому все артефакты одной проверки связаны между собой. Обработка таймаутов, ошибок соединения и повреждённого JSON вместе с кодами завершения позволяет подключить утилиту к планировщику и системе мониторинга кампуса.

4. Снимок экрана

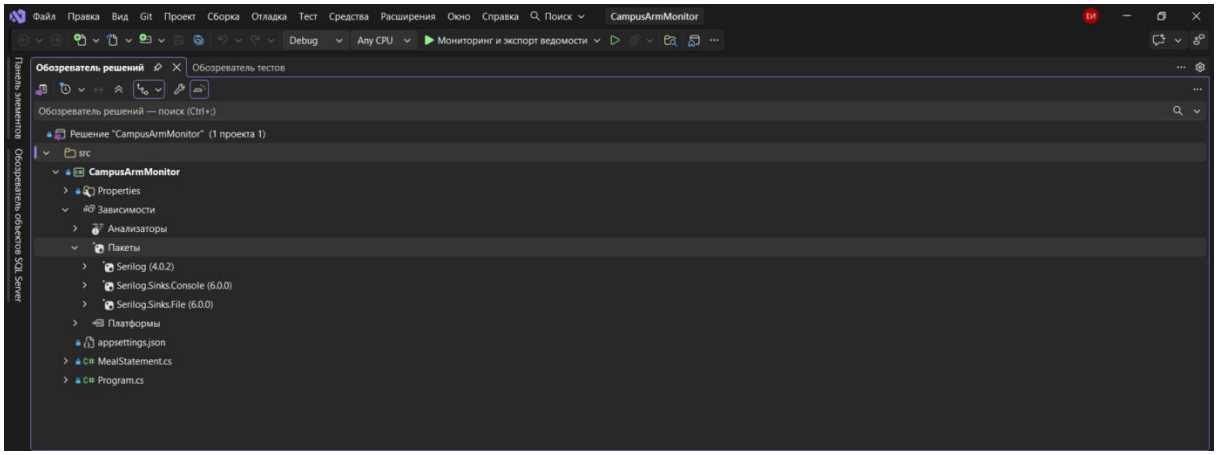


Рисунок 1 – Проект CampusArmMonitor в Visual Studio: Program.cs, пакеты Serilog

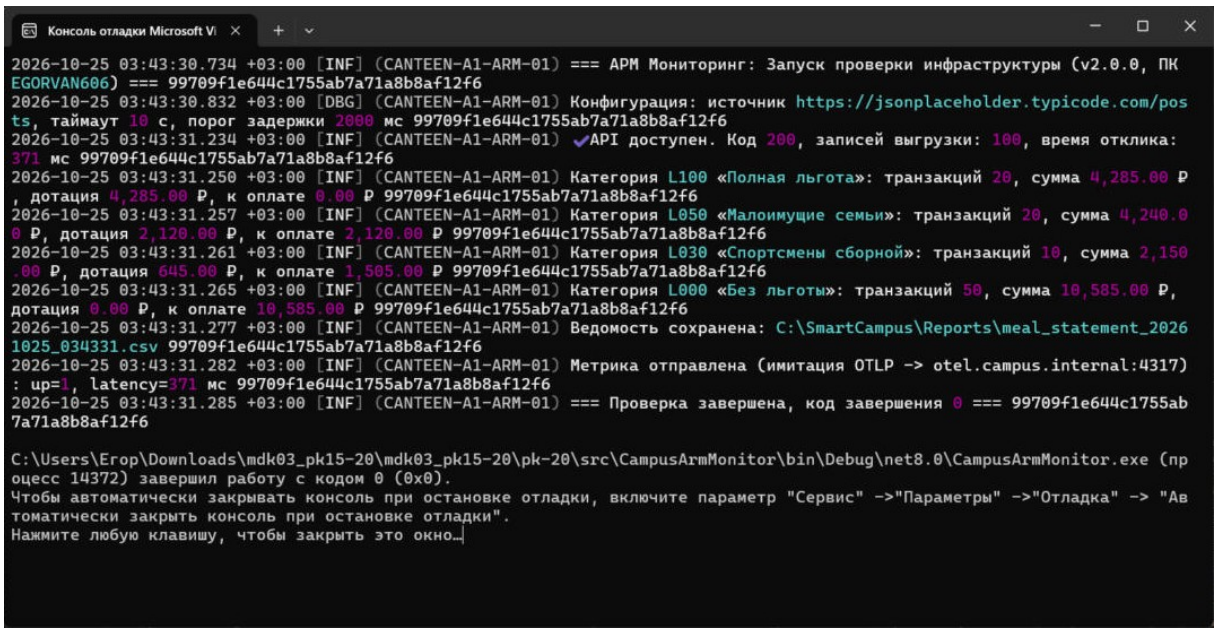


Рисунок 2 – Запуск АРМ-монитора: API доступен, время отклика, ведомость по категориям льгот, отправка метрики

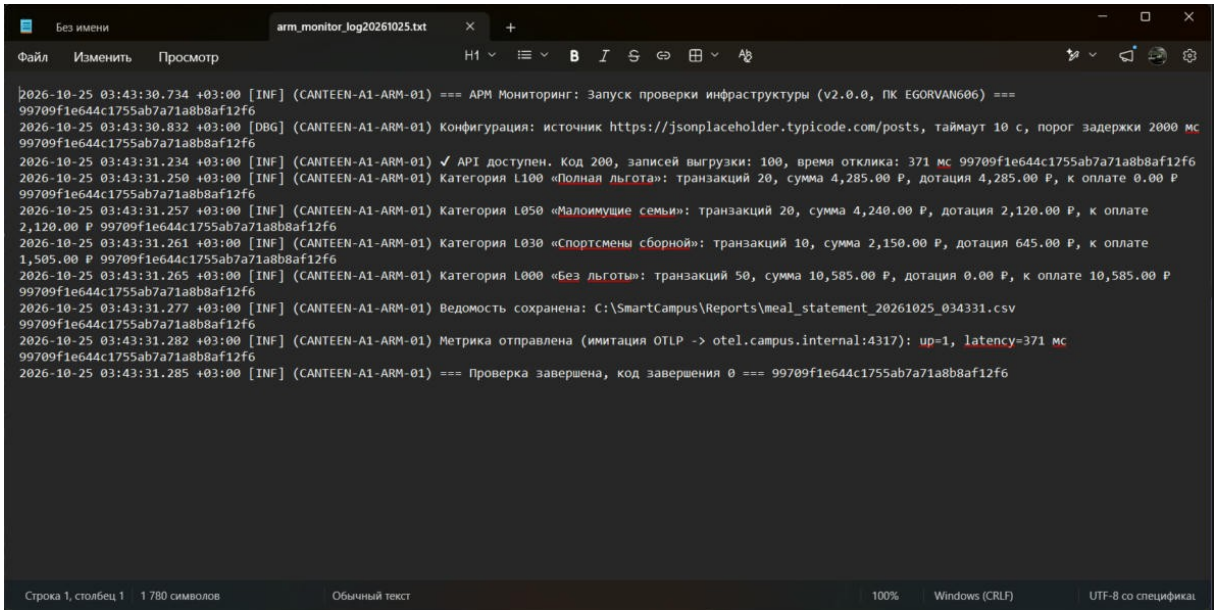


Рисунок 3 – Файл лога Serilog C:\SmartCampus\Logs\arm_monitor_log<дата>.txt со StationId и TraceId

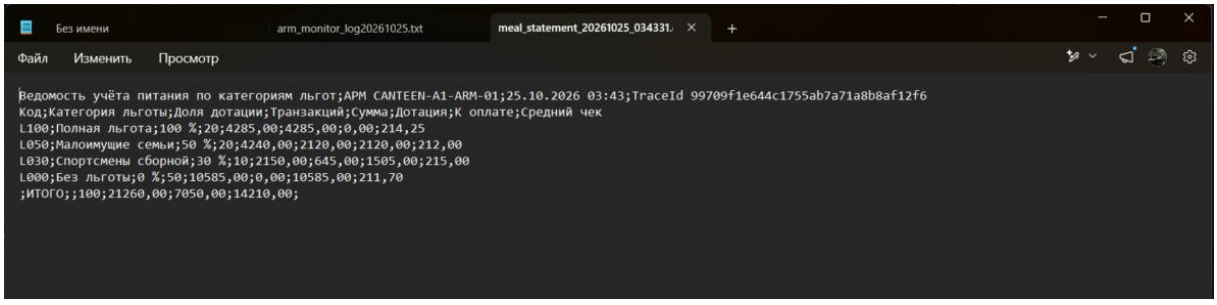


Рисунок 4 – Сформированная ведомость C:\SmartCampus\Reports\meal_statement_….csv

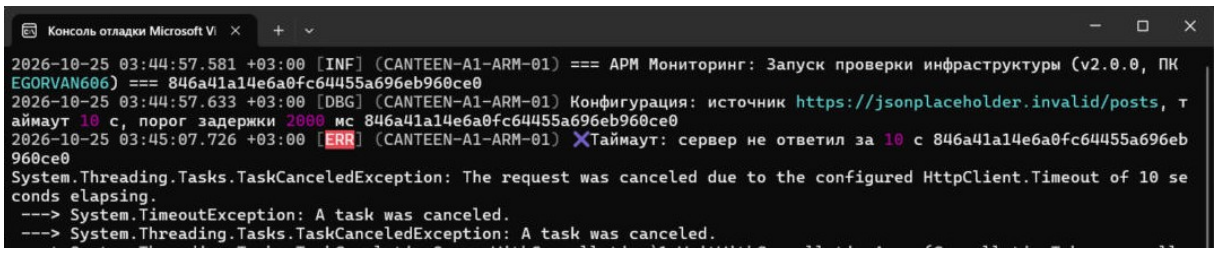


Рисунок 5 – Сбой: API недоступен — таймаут 10 с записан в лог Serilog с уровнем ERR и стеком исключения

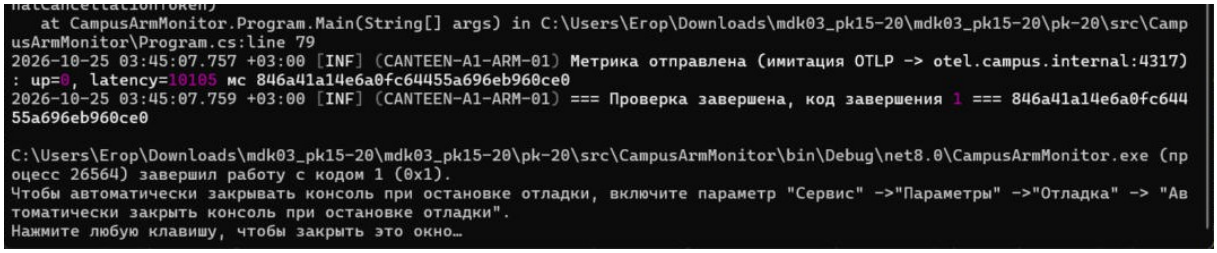


Рисунок 6 – Окончание проверки при сбое: метрика arm_api_up = 0, код завершения 1